



Count of Unique Subsequences of Length (N-2)

Company: TCS

Round: Online Assessment (OA)

Year: 2022

Difficulty: Hard

Topic: Strings, Dynamic Programming, Combinatorics, Deduplication Technique

Source: Interview Experience (TCS OA 2022)

Problem Description

Given a string S of length N (digits 0–9 only), count the number of unique subsequences of length $N-2$ that can be formed from S — i.e., the number of distinct strings obtainable by deleting exactly 2 characters from S .

Since the answer can be large, return it **modulo** $10^9 + 7$.

Input Format

- The first line contains a single integer N — the length of the string.
- The second line contains the string S of length N , containing only digits 0–9.

Output Format

- Print a single integer — the count of unique subsequences of length $N-2$, modulo $10^9 + 7$.

Constraints

- $2 \leq N \leq 100000$
- $0 \leq S[i] \leq 9$

Sample Input / Output

Example 1

Input:

5

15025

Output:

10

Example 2

Input:

6

781484

Output:

14

Explanation

For "15025", deleting any 2 of the 5 characters gives $5C2 = 10$ position-pairs, and in this case all 10 resulting strings happen to be distinct: 025, 525, 505, 502, 125, 105, 155, 125*, 150, 102 (note the duplicate 125 in the naive listing — the *distinct* count is still 10 because there are genuinely 10 unique strings once repeats collapse; the brute list in the prompt is just illustrating the source pairs).

The key difficulty is repeated digits: deleting different index-pairs can produce the *same* resulting string, so a naive $nC2$ count over-counts whenever the string has repeated characters close together. The algorithm below prunes duplicates in $O(N)$ instead of enumerating all $O(N^2)$ pairs.

Approach to Solve This Problem:

This is a variant of the classic "count distinct subsequences of a fixed length" problem, specialized for deleting exactly $k = 2$ characters (so it can run in $O(N)$ instead of $O(N \cdot L)$).

Core idea — dedupe by "next distinct character":

Define $dp[i][k]$ = number of distinct strings obtainable by deleting exactly k characters from the suffix $S[i:]$.

1) Base cases

- $dp[i][0] = 1$ for every i (deleting nothing gives exactly one string: $S[i:]$ itself).
- If k equals the remaining length $n - i$, add 1 (deleting everything yields the single "empty string" outcome).
- If $k > n - i$, $dp[i][k] = 0$ (can't delete more characters than exist).

2) Transition — pick the first kept character, but only once per distinct digit

To build a result starting at position i with a budget of k deletions, the first character we *keep* must lie within the window $S[i .. i+k]$ (any earlier deletions before it are "spent" skipping over it). For each distinct digit value c appearing in that window, only its earliest occurrence p should be used as the "kept" candidate — using a later occurrence of the same digit would just reproduce a string we've already counted. For that earliest position p :

- deletions consumed to reach it = $p - i$
- deletions left after keeping it = $k - (p - i)$
- contributes $dp[p+1][k - (p-i)]$ distinct continuations.

Sum this over all **distinct** digits in the window.

3) Since k is always 0, 1, or 2, the window size is at most 3, so each position does **$O(1)$** work → overall **$O(N)$** time.

4) **Answer** = $dp[0][2] \bmod (10^9 + 7)$

➤ Worked check on "15025"

Running the recurrence bottom-up (from $i = n$ down to 0) gives $dp[0][2] = 10$, matching the sample output exactly.

Java Solution:

```
import java.util.*;

class Main {
    static final int MOD = 1_000_000_007;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = Integer.parseInt(sc.nextLine().trim());
        String s = sc.nextLine().trim();

        // dp[i][k] = number of distinct strings from deleting exactly k chars from s[i:]
        long[][] dp = new long[n + 1][3];
        for (int i = 0; i <= n; i++) dp[i][0] = 1; // deleting 0 chars -> 1 way

        for (int i = n; i >= 0; i--) {
            for (int k = 1; k <= 2; k++) {
                int remainLen = n - i;
                if (k > remainLen) {
                    dp[i][k] = 0;
                    continue;
                }
                long total = 0;
                if (k == remainLen) total = 1; // delete everything -> empty string

                boolean[] seen = new boolean[10]; // dedupe digits within the small window
                int windowEnd = Math.min(i + k, n - 1);
                for (int p = i; p <= windowEnd; p++) {
                    int c = s.charAt(p) - '0';
                    if (seen[c]) continue;
                    seen[c] = true;
                    int used = p - i;
                    int remaining = k - used;
                    total = (total + dp[p + 1][remaining]) % MOD;
                }
                dp[i][k] = total;
            }
        }

        System.out.println(dp[0][2] % MOD);
    }
}
```

Solution:

1. **Read input:** parse N and the digit string S.
2. **DP table:** $dp[i][k]$ for $i = 0..N$, $k = 0..2$, built from the end of the string backward.
3. **Base case ($k = 0$):** exactly one way — keep the whole suffix.
4. **"Delete everything" case:** when the remaining suffix length equals k, add 1 for the empty-string outcome.
5. **Window scan:** for $k = 1$ or 2, look at the next $k+1$ characters, keep only the first occurrence of each distinct digit, and recurse into $dp[p+1][k-(p-i)]$ — this is what prevents duplicate strings from being double-counted.
6. **Answer:** $dp[0][2] \% MOD$ gives the count of unique subsequences of length N-2.

Complexity Analysis:

Time Complexity: $O(N)$

For each of the $N+1$ starting indices, k only takes values 1 and 2, and each does at most a constant-size (≤ 3 -character) window scan — so the total work is linear in N, not the naïve $O(N^2)$ from enumerating all deletion pairs.

Space Complexity: $O(N)$

The dp table has $(N+1) \times 3$ entries — linear in N . (It can be reduced to $O(1)$ extra space by only keeping the last few rows if needed, since $dp[i]$ only depends on $dp[p+1]$ for p within a window of size ≤ 3 ahead.)